# PROYECTO 1:
# ARQUITECTURA CLIENTE-SERVIDOR Y CONSUMO DE API REST (FASTAPI & JAVASCRIPT)

**Estudiante:** Patricia Jaime
**Profesor:** Diego Alberto Ayala Vázquez
**Asignatura:** Programación IV

6 de octubre de 2026

| | |
|---|---|
| **Repositorio GitHub:** | https://github.com/patriciajaime/tienda-fastapi-frontend |
| **Tecnologías:** | Python 3, FastAPI, Uvicorn, JavaScript (Fetch API), HTML5, CSS3 |
| **Arquitectura:** | Cliente-Servidor Distribuida (Puertos 8000 y 3000) |

# 1. Introducción

El presente reporte documenta el desarrollo y verificación de un sistema web basado en una arquitectura cliente-servidor distribuida. Se implementó una API REST backend utilizando **FastAPI** ejecutándose en el puerto `8000`, y un cliente frontend estático servido mediante el módulo `http.server` de Python en el puerto `3000`. La comunicación asíncrona entre ambas capas se realizó a través del API `fetch()` de JavaScript, manejando peticiones HTTP (`GET`, `POST`, `DELETE`) y gestionando el intercambio de datos en formato JSON bajo políticas de CORS (*Cross-Origin Resource Sharing*).

# 2. Arquitectura y Estructura del Proyecto

## 2.1. Estructura de Directorios

La estructura del proyecto fue organizada manteniendo una clara separación entre el backend y el frontend:

- `tienda_proyecto/` (Directorio Raíz)
  - `backend/`
    - `main.py` – Servidor FastAPI con endpoints (`GET`, `POST`, `DELETE`) y middleware.
  - `frontend/`
    - `index.html` – Interfaz web estática y cliente JavaScript (`fetch` API).
  - `.gitignore` – Exclusión de archivos temporales de Python y del sistema.

## 2.2. Tecnologías Utilizadas

- **Backend:** Python 3, FastAPI, Uvicorn (Puerto `8000`).
- **Frontend:** HTML5, CSS3, JavaScript ES6 (`fetch` API) (Puerto `3000`).
- **Control de Versiones:** Git y GitHub.

# 3. Descripción de Endpoints y Métodos HTTP

1. `GET /productos`**:** Retorna el listado completo de productos en formato JSON.
2. `POST /productos`**:** Recibe un objeto JSON con la información del nuevo producto (`id`, `nombre`, `precio`) y lo agrega a la estructura de datos.
3. `DELETE /productos/{producto_id}`**:** Elimina el producto cuyo ID coincida con el parámetro enviado en la URL.
4. **Middleware** `X-Process-Time`**:** Mide el tiempo exacto de ejecución de cada solicitud HTTP y lo adjunta en los encabezados (*headers*) de la respuesta.

# 4. Evidencias de Ejecución y Funcionamiento

## 4.1. Evidencia 1: Petición GET (Carga Inicial de Productos)

Demostración de la interfaz gráfica al iniciar la aplicación en `http://localhost:3000`. La tabla consume dinámicamente los datos expuestos por el backend en el puerto `8000`.

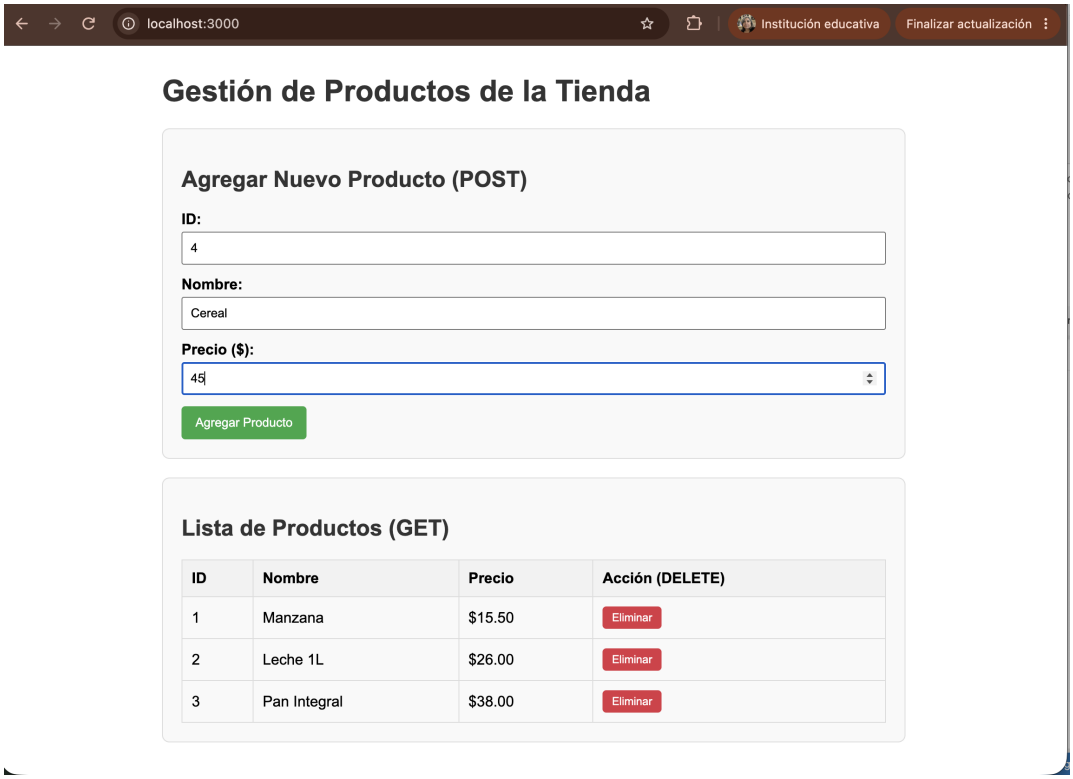



Figura 1: Carga inicial de productos vía petición GET.

## 4.2. Evidencia 2: Petición POST (Inserción de Nuevo Producto)

Demostración de la creación de un nuevo registro a través del formulario de la interfaz. Se envía un objeto JSON mediante `fetch()` con método `POST`.

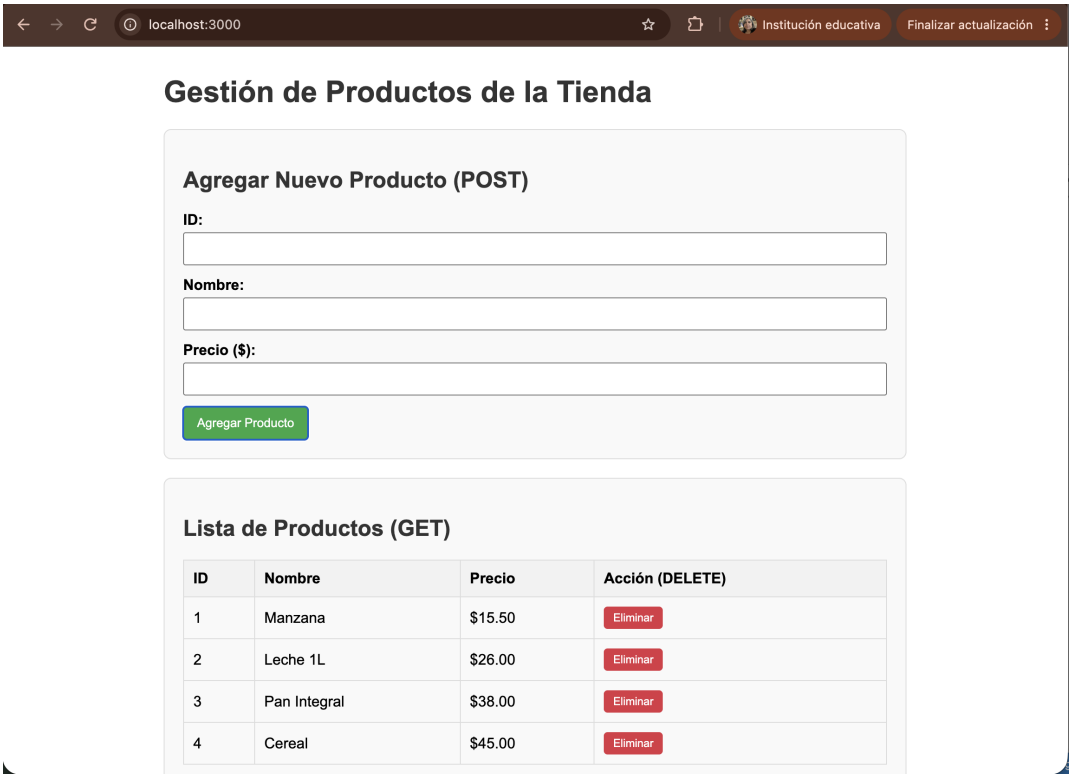



Figura 2: Inserción de nuevo producto procesado dinámicamente vía POST.

### 4.3. Evidencia 3: Petición DELETE (Eliminación de Registro)

Demostración del borrado de un registro específico seleccionando el botón .Eliminar". La petición HTTP `DELETE` actualiza el estado del servidor y sincroniza la vista en el cliente.

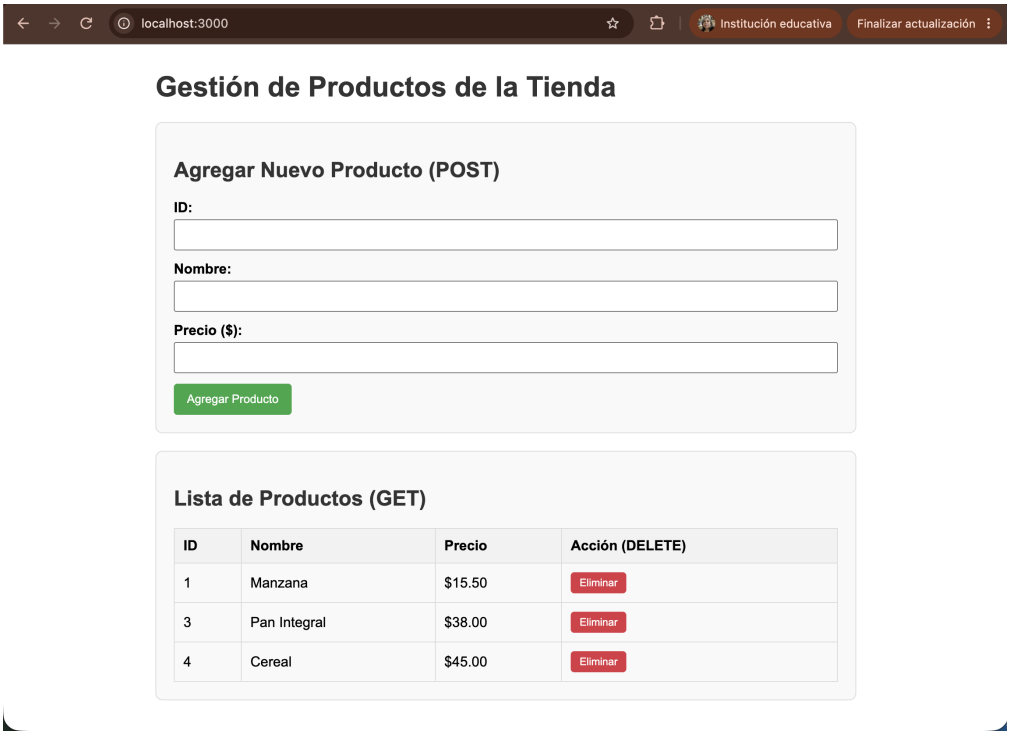



Figura 3: Actualización de la tabla tras eliminar un registro vía DELETE.

## 5. Conclusiones

- Se logró implementar satisfactoriamente una arquitectura desacoplada, separando la lógica del servidor de la capa de presentación.
- El uso de CORS en FastAPI garantizó que el cliente ubicado en un origen distinto (`localhost:3000`) pudiera consumir de forma segura los recursos del servidor (`localhost:8000`).
- El uso del API `fetch()` permitió actualizar dinámicamente el DOM de la aplicación sin necesidad de recargar la página completa, ofreciendo una experiencia de usuario fluida y asíncrona.